

Báo cáo Bài tập trên lớp 01

Họ và tên: Nguyễn Thành Nam

MSSV: 20225368

Bài tập 01.01 – Đóng gói dữ liệu

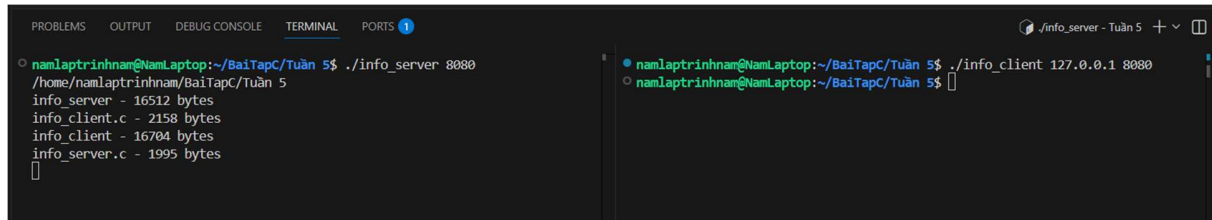
- Ứng dụng info_client lấy tên thư mục hiện tại, danh sách các tập tin và kích thước (trên máy client). Các dữ liệu này sau đó được đóng gói và chuyển sang info_server.
- Ứng dụng info_server nhận dữ liệu từ info_client, tách các dữ liệu và in ra màn hình.
- Yêu cầu kích thước dữ liệu được gửi là nhỏ nhất(không truyền các dữ liệu dư thừa).

Ví dụ:

/mnt/c/Users/admin/source/baitap

bai1.c – 12871 bytes

bai2.c – 1437 bytes



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1
○ namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./info_server 8080
/home/namlaptrinhnam/BaiTapC/Tuần 5
info_server - 16512 bytes
info_client.c - 2158 bytes
info_client - 16704 bytes
info_server.c - 1995 bytes
○ namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./info_client 127.0.0.1 8080
○ namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$
```

Hình 1: Minh họa hoạt động truyền nhận và bóc tách dữ liệu thư mục giữa Client và Server trong Bài 01.01.

Giải thích chi tiết (Usecase thử nghiệm): Kịch bản kiểm tra khả năng thu thập và đóng gói dữ liệu tối ưu:

- **Tại Client (Cửa sổ bên phải):** Chương trình info_client chạy và lập tức quét thư mục làm việc hiện tại (/home/namlaptrinhnam/BaiTapC/Tuần 5). Nó thu thập đường dẫn tuyệt đối, đếm số lượng file hợp lệ và lấy kích thước từng file. Để tối ưu bằng thông theo yêu cầu đề bài, toàn bộ dữ liệu này được Client đóng gói dưới dạng nhị phân nguyên thủy (binary) thay vì gửi chuỗi văn bản dài dòng, sau đó truyền qua TCP socket.
- **Tại Server (Cửa sổ bên trái):** Chương trình info_server nhận gói dữ liệu nhị phân từ Client. Nhờ đã quy ước trước cấu trúc gói tin (độ dài đường dẫn -> đường dẫn -> số lượng file -> [độ dài tên -> tên -> kích thước]...), Server tiến hành bóc tách thành

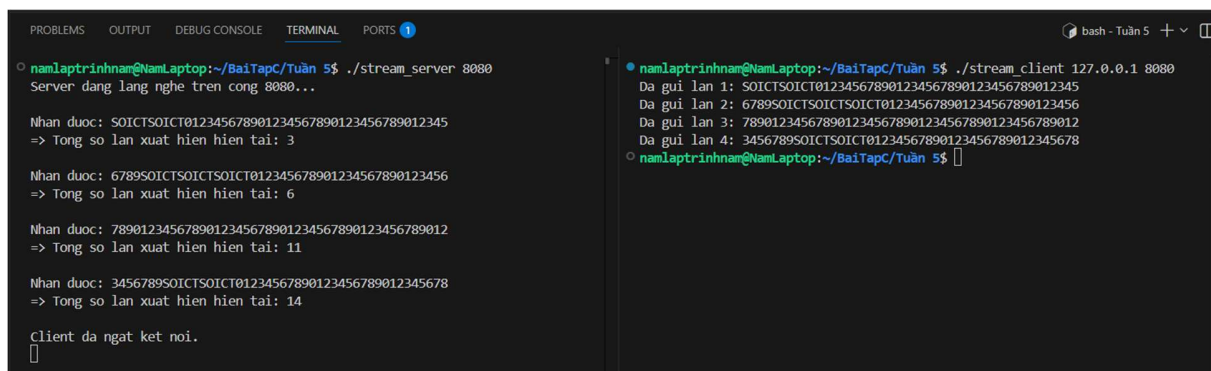
công và in ra màn hình đúng định dạng yêu cầu: hiển thị đường dẫn thư mục, theo sau là danh sách các tập tin kèm theo dung lượng tính bằng byte.

Bài tập 01.02 – Xử lý dữ liệu streaming

- Ứng dụng client liên tục gửi dữ liệu văn bản sang server.
- Ứng dụng server nhận dữ liệu từ client, cập nhật và in ra màn hình số lần xuất hiện xâu ký tự “0123456789” với dữ liệu đã nhận được.
- Cần xử lý trường hợp khi xâu “0123456789” nằm giữa 2 lần truyền.

Ví dụ dữ liệu gửi từ client sau 4 lần (không bao gồm ký tự xuống dòng):

```
SOICTSOICT012345678901234567890123456789012345
6789SOICTSOICTSOICT012345678901234567890123456
7890123456789012345678901234567890123456789012
3456789SOICTSOICT01234567890123456789012345678
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./stream_server 8080
Server đang lắng nghe trên cổng 8080...

Nhan duoc: SOICTSOICT01234567890123456789012345
=> Tong so lan xuat hien hien tai: 3

Nhan duoc: 6789SOICTSOICTSOICT012345678901234567890123456
=> Tong so lan xuat hien hien tai: 6

Nhan duoc: 7890123456789012345678901234567890123456789012
=> Tong so lan xuat hien hien tai: 11

Nhan duoc: 3456789SOICTSOICT01234567890123456789012345678
=> Tong so lan xuat hien hien tai: 14

Client da ngat ket noi.
[]

namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./stream_client 127.0.0.1 8080
Da gui lan 1: SOICTSOICT012345678901234567890123456789012345
Da gui lan 2: 6789SOICTSOICTSOICT012345678901234567890123456
Da gui lan 3: 7890123456789012345678901234567890123456789012
Da gui lan 4: 3456789SOICTSOICT01234567890123456789012345678
namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$
```

Hình 2: Minh họa kết quả xử lý luồng dữ liệu (streaming) và đếm chuỗi "0123456789" bị phân mảnh qua nhiều lần gửi trong Bài 01.02.

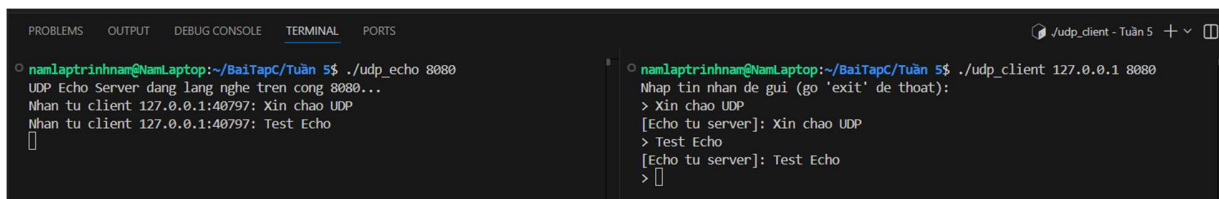
Giải thích chi tiết (Usecase thử nghiệm): Kịch bản này kiểm chứng khả năng xử lý ranh giới phân mảnh gói tin đặc trưng của giao thức TCP:

- **Quá trình gửi (Bên phải):** Client thực hiện 4 lần gửi luồng văn bản liên tiếp. Đặc biệt, ở lần gửi 1 kết thúc bằng chuỗi "012345", và lần gửi 2 bắt đầu bằng "6789". Chuỗi mục tiêu "0123456789" đã cố tình bị cắt làm đôi vắt ngang giữa hai lần truyền.
- **Quá trình nhận và xử lý (Bên trái):** Server nhận tuần tự từng khối dữ liệu và in ra màn hình. Nhờ thuật toán duy trì bộ đệm chứa 9 ký tự cuối của lần nhận trước để ghép nối với lần nhận sau, Server đã không bỏ sót bất kỳ chuỗi mục tiêu nào. Điều này thể hiện rõ qua biến đếm (Tổng số lần xuất hiện hiện tại):
 - Lần 1: Đếm được 3.

- Lần 2: Đếm được 6 (Ghi nhận thành công chuỗi bị cắt đôi ở ranh giới lần 1 và lần 2, cộng thêm 2 chuỗi trọn vẹn ở lần 2).
- Lần 3 và 4: Biến đếm tiếp tục tăng chính xác lên 11 và 14. Kết quả hoàn toàn khớp với dữ liệu đầu vào thực tế.

Bài tập 01.03 – Ứng dụng chat sử dụng UDP

- Ứng dụng `udp_echo` nhận dữ liệu và phản hồi lại chính dữ liệu đó cho bên truyền



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
○ namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./udp_echo 8080
UDP Echo Server đang lắng nghe trên cổng 8080...
Nhận từ client 127.0.0.1:40797: Xin chào UDP
Nhận từ client 127.0.0.1:40797: Test Echo
[]

○ namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 5$ ./udp_client 127.0.0.1 8080
Nhập tin nhắn để gửi (gõ 'exit' để thoát):
> Xin chào UDP
[Echo từ server]: Xin chào UDP
> Test Echo
[Echo từ server]: Test Echo
> []
  
```

Hình 3: Minh họa quá trình truyền nhận thông điệp hai chiều sử dụng giao thức UDP giữa `udp_client` và `udp_echo` server.

Giải thích chi tiết (Usecase thử nghiệm): Kịch bản mô phỏng hoạt động của mô hình không kết nối (Connectionless):

- **Thiết lập Server (Bên trái):** `udp_echo` server được khởi chạy, bind (gắn kết) vào cổng 8080 và ở trạng thái chờ gói tin datagram bằng hàm `recvfrom()`.
- **Giao tiếp hai chiều:** Tại Client (bên phải), người dùng gửi hai thông điệp là "Xin chào UDP" và "Test Echo".
 - Khi Server nhận được gói tin, nó trích xuất chính xác địa chỉ IP và cổng của Client (127.0.0.1:40797) từ header của UDP datagram, sau đó in ra màn hình. Ngay lập tức, Server dùng hàm `sendto()` để "dội" (echo) chính xác chuỗi dữ liệu đó ngược lại địa chỉ Client vừa trích xuất.
 - Ở phía Client, ứng dụng cũng liên tục lắng nghe và đã nhận lại nguyên vẹn các phản hồi từ Server, hiển thị dưới dạng "[Echo từ server]: ...". Quá trình diễn ra tức thời, minh chứng cho sự gọn nhẹ và tốc độ của UDP.